
Chapter 4

Intermediate SQL



Outline

- ▶ **Join Expressions**
- ▶ **Views**
- ▶ **Transactions**
- ▶ **Integrity Constraints**
- ▶ **SQL Data Types and Schemas**
- ▶ **Authorization**



Join Expressions

- ▶ **Join operations** take two relations and return as a result another relation.
- ▶ A join operation is a **Cartesian product** which requires that tuples in the two relations *match* (under some condition). It also specifies the attributes that are present in the result of the join.
- ▶ The join operations are typically used as *sub-query expressions in the from clause*.



Join Conditions

- ▶ The **on condition** allows a *general predicate* over the relations being joined.

```
select *  
from student join takes on student.ID= takes.ID;
```

- ▶ The above query is equivalent to the following query:

```
select *  
from student, takes  
where student.ID= takes.ID;
```

- ▶ A **join expressions** using the **on** condition can express *a richer class of join conditions* than natural join.



Outer Joins

- ▶ Suppose we wish to display a list of all *students*, displaying their *ID*, and *name*, *dept_name*, and *tot_cred*, along with the *courses* that they have taken.

```
select *  
from student natural join takes;
```

- ▶ Suppose that there is *some student who takes no courses*. We would thus not see any information about students who have not taken any courses.
- ▶ More generally, **some tuples in either or both of the relations being joined may be “lost” in this way.**



Join operations – Example

- ▶ Relation *course*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- ▶ Relation *prereq*

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- ▶ Observe that *prereq* information is missing for CS-315 and *course* information is missing for CS-437

select *

from course natural join prereq

- ▶ The course CS-315 and CS-437 would not appear in the result.

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190



Outer Joins

- ▶ The **outer join** operation works in a manner that preserve those tuples that would be lost in a join, by *creating tuples in the result containing null values*.
- ▶ There are in fact three forms of outer join:
 - ▶ The **left outer join** preserves tuples only in the relation named before (to the left of) the *left outer join* operation.
 - ▶ The **right outer join** preserves tuples only in the relation named after (to the right of) the *right outer join* operation.
 - ▶ The **full outer join** preserves tuples in *both* relations.
- ▶ The join operations we studied earlier that do not preserve non-matched tuples are called **inner join operations**.



Outer join

- ▶ Compute the **left outer-join** operation as follows:
 - ▶ First, compute the result of the **inner join** as before.
 - ▶ Second, add a tuple r to the result, for every tuple t in the **left-hand-side relation** that **does not match any tuple** in the **right-hand-side relation**, as follows:
 - ▶ The attributes of tuple r that are derived from the left-hand-side relation are filled in with the values from tuple t .
 - ▶ The remaining attributes of r are filled with **null** values.



Left Outer Join

- ▶ The **left outer join** preserves tuples only in the relation named before (to the left of) the left outer join operation.

select *

from *course* **natural left outer join** *prereq*

<i>course</i>			
<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

**natural left
outer join**

<i>prereq</i>	
<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>



Right Outer Join

- ▶ The **right outer join** preserves tuples only in the relation named after (to the right of) the right outer join operation.

select *

from *course* **natural right outer join** *prereq*

<i>course</i>			
<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

**natural right
outer join**

<i>prereq</i>	
<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101



<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



Full Outer Join

- ▶ The **full outer join** preserves tuples in both relations.

select *

from *course* **natural full outer join** *prereq*

<i>course</i>			
<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

**Natural full
outer join**

<i>prereq</i>	
<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101



<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



Join Types and Conditions

- ▶ **Join condition** – defines *which tuples* in the two relations match, and *what attributes* are present in the result of the join.
- ▶ **Join type** – defines *how tuples* in each relation that *do not match* any tuple in the other relation (based on the join condition) are treated.
- ▶ The default join type, when the join clause is used without the outer prefix is the inner join.

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join conditions</i>
natural
on <predicate>
using ($A_1, A_2, \dots, A_n$)



Views

- ▶ It is not desirable for all users to see the **entire logical model** (all the actual relations stored in the database)
- ▶ *Security considerations*
 - ▶ Consider a clerk who needs to know an *instructor's ID, name and department*, but does not have authorization to see the *instructor's salary* amount. This person should see a relation described, in SQL, by

```
select ID, name, dept_name
from instructor
```
- ▶ May wish to create a *personalized collection of relations* that is better matched to a certain user's intuition than is the logical model.



Views

- ▶ Suppose we want to have a list of all *course* sections offered by the Physics *department* in the Fall 2009 semester, with the *building* and *room number* of each *section*.

```
select course.course_id, sec_id, building, room_number
from course, section
where course.course_id = section.course_id
       and course.dept_name = 'Physics'
       and section.semester = 'Fall'
       and section.year = '2009';
```

- ▶ It is possible to **compute and store the results** of the above queries and then make the stored relations available to users. ***Generally this is a bad idea.***



Views

- ▶ SQL allows a “*virtual relation*”, called a **view**, to be defined by a query, and the relation conceptually contains the result of the query.
- ▶ The virtual relation is **NOT precomputed and stored**, but instead is computed by *executing the query whenever the virtual relation is used*.



View Definition

- ▶ A view is defined using the **create view** command
create view v as < query expression >;
where <query expression> is any legal SQL expression.
The **view name** is represented by v .
- ▶ Once a view is defined, the **view name** can be used to refer to the virtual relation that the view generates.
- ▶ View definition is **NOT the same** as **creating a new relation by evaluating the query expression**.
- ▶ Rather, a view definition causes the **saving of an expression**; the expression is **substituted into queries using the view**.



Example Views

- ▶ A **view** of *instructors* without their *salary*

create view *faculty* **as**

select *ID, name, dept_name*

from *instructor*

- ▶ A **view** that lists all *course sections* offered by the Physics department in the Fall 2009 semester with the *building* and *room number* of each section

create view *physics_fall_2009* **as**

select *course.course_id, sec_id, building, room_number*

from *course, section*

where *course.course_id = section.course_id*

and *course.dept_name = 'Physics'*

and *section.semester = 'Fall'*

and *section.year = '2009';*

- ▶ Create a **view** of *department salary* totals

create view *departments_total_salary* (*dept_name, total_salary*) **as**

select *dept_name, sum (salary)*

from *instructor*

group by *dept_name;*



Using Views in SQL Queries

- ▶ Once we have defined a view, we can use the **view name** to refer to the virtual relation that the view generates.
- ▶ Using the view `physics_fall_2009`, we can *find all Physics courses offered in the Fall 2009 semester in the Watson building* by writing:

```
select course_id
from physics_fall_2009
where building= 'Watson';
```

- ▶ When we define a view, the database system **stores the definition of the view itself**, rather than the *result of evaluation of the query expression that defines the view*.
- ▶ Wherever a view relation appears in a query, it is *replaced by the stored query expression*.



Views Defined Using Other Views

- ▶ One view may be used in the expression **defining another view**.
- ▶ A view relation v_1 is said to *depend directly* on a view relation v_2 if v_2 is used in the expression defining v_1 .
- ▶ A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2 .
- ▶ A view relation v is said to be *recursive* if it *depends on itself*.



Views Defined Using Other Views

- ▶ create view *physics_fall_2009* as
select *course.course_id, sec_id, building, room_number*
from *course, section*
where *course.course_id = section.course_id*
and *course.dept_name = 'Physics'*
and *section.semester = 'Fall'*
and *section.year = '2009'*;
- ▶ create view *physics_fall_2009_watson* as
select *course_id, room_number*
from *physics_fall_2009*
where *building= 'Watson'*;



View Expansion

- ▶ A way to define the meaning of *views defined in terms of other views*.
- ▶ Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- ▶ **View expansion** of an expression repeats the following replacement step:
 - repeat**
 - Find any view relation v_i in e_1
 - Replace the view relation v_i by the expression defining v_i
 - until** no more view relations are present in e_1
- ▶ As long as the view definitions are not recursive, this loop will terminate



View Expansion

- ▶ The above example is equivalent to:
create view *physics_fall_2009_watson* **as**
(**select** *course_id, room_number*
from (**select** *course.course_id, building, room_number*
 from *course, section*
 where *course.course_id = section.course_id*
 and *course.dept_name = 'Physics'*
 and *section.semester = 'Fall'*
 and *section.year = '2009'*)
where *building = 'Watson'*;



Update of a View

- ▶ **Add** a new tuple to *faculty* view

```
create view faculty as  
select ID, name, dept_name  
from instructor;
```

insert into *faculty* values ('30765', 'Green', 'Music');

- ▶ This insertion must be represented by the insertion of the tuple

('30765', 'Green', 'Music', ***null***)

into the *instructor* relation

← salary



Some Updates cannot be Translated Uniquely

```
create view instructor_info as  
  select ID, name, building  
  from instructor, department  
  where instructor.dept_name= department.dept_name;  
  
insert into instructor_info values ('69987', 'White', 'Taylor');
```

- ▶ Suppose there is no *instructor* with *ID* 69987, and no *department* in the *Taylor* building.
- ▶ Then the only possible method is to insert ('69987', 'White', *null*, *null*) into *instructor* and (*null*, 'Taylor', *null*) into *department*.



Some Updates cannot be Translated Uniquely

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	null	null

instructor

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Electrical Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
null	Taylor	null

department

- ▶ However, this update does *not have the desired effect*, since the view relation *instructor* info still does **NOT** include the tuple ('69987', 'White', 'Taylor').
- ▶ Thus, there is no way to update the relations *instructor* and *department* by using *nulls* to get the desired update on *instructor* info .
- ▶ Because of problems such as these, *modifications are generally not permitted on view relations, except in limited cases.*



Conditions on Updatability

- ▶ A SQL view is said to be **updatable** if the following conditions are *all satisfied* by the query defining the view:
 - ▶ The **from** clause has *only one database relation*.
 - ▶ The **select** clause contains *only attribute names of the relation*, and *does not have any expressions, aggregates, or distinct specification*.
 - ▶ Any attribute *not listed* in the **select** clause can be set to *null*
 - ▶ The query *does not* have a **group** by or **having** clause.
- ▶ Under these constraints, the *update*, *insert*, and *delete* operations would be allowed on the following view:

```
create view history_instructors as
select *
from instructor
where dept_name= 'History';
```



With check option

```
create view history_instructors as  
select *  
from instructor  
where dept_name= 'History';
```

- ▶ Suppose that a user tries to **insert** the tuple ('25566', 'Brown', 'Biology', 100000) into the *history_instructors* view.
- ▶ This tuple can be inserted into the *instructor* relation, but it would not appear in the *history_instructors* view since it **does not satisfy the selection imposed by the view**.
- ▶ By default, SQL would allow the above update to proceed.
- ▶ However, views can be defined with a **with check option** clause: if a tuple inserted into the view does not satisfy the view's where clause condition, the insertion is **rejected** by the database system.

```
create view history_instructors as  
select *  
from instructor  
where dept_name= 'History'  
with check option;
```



Materialized Views

- ▶ Certain database systems allow view relations to be stored, but they make sure that, *if the actual relations used in the view definition change*, the view is kept up-to-date. Such views are called **Materialized Views**.
- ▶ If relations used in the query are updated, the materialized view result becomes out of date
 - ▶ Need to maintain the view, by *updating the view whenever the underlying relations are updated*.
 - ▶ Some database systems, however, *perform view maintenance lazily, when the view is accessed*.
- ▶ **Applications may benefit if the view is materialized**
 - ▶ Applications that use a view frequently.
 - ▶ Applications that demand fast response to certain queries that compute aggregates over large relations



Transactions

- ▶ A **transaction** consists of a sequence of query and/or update statements.
- ▶ The SQL standard specifies that *a transaction begins implicitly when an SQL statement is executed*. One of the following SQL statements must end the transaction:
 - ▶ **Commit work** commits the current transaction; i.e. makes the updates performed by the transaction become permanent in the database.
 - ▶ **Rollback work** causes the current transaction to be rolled back; i.e. undoes all the updates performed by the SQL statements in the transaction.
- ▶ **Transaction rollback** is useful if some error condition is detected during execution of a transaction.



Integrity Constraints

- ▶ **Integrity constraints** guard against *accidental damage* to the database, by ensuring that authorized changes to the database do not result in a loss of *data consistency*.
 - ▶ An *instructor* name cannot be *null*.
 - ▶ No two instructors can have the same *instructor ID*.
 - ▶ Every *department* name in the *course* relation must have a matching *department* name in the *department* relation.
 - ▶ The *budget* of a *department* must be greater than \$0.00.
- ▶ Integrity constraints are usually declared as part of the **create table** command used to create relations.
- ▶ However, integrity constraints can also be added to an existing relation by using the command **alter table** *table-name* **add constraint**.



Constraints on a Single Relation

- ▶ **not null**
- ▶ **primary key**
- ▶ **unique**
- ▶ **check (P)**, where P is a predicate



Not Null Constraint

▶ Not Null

- ▶ Declare *name* and *budget* to be **not null**

name **varchar**(20) **not null**

budget **numeric**(12,2) **not null**

- ▶ The **not null** specification *prohibits the insertion of a null value for the attribute*.
- ▶ In particular, SQL prohibits *null* values in the primary key of a relation schema.
- ▶ If an attribute is declared as the primary key, it would not need to be declared explicitly to be **not null**.



Unique Constraint

- ▶ **unique** ($A_1, A_2, \dots, A_m$)
 - ▶ The unique specification states that the attributes $A_1, A_2, \dots, A_m$ form a **candidate key**.
 - ▶ That is, no two tuples in the relation can be equal on *all the listed attributes*.
 - ▶ **Candidate keys are permitted to be *null*** (in contrast to primary keys).
 - ▶ A *null* value does not equal any other value.



The check Clause

- ▶ **check** (*P*)

where *P* is a predicate that *must be satisfied* by every tuple in a relation.

- ▶ Example: Ensure that *semester* is one of *fall*, *winter*, *spring* or *summer*:

```
create table section
```

```
  (course_id      varchar (8),
```

```
   sec_id        varchar (8),
```

```
   semester      varchar (6),
```

```
   year          numeric (4,0),
```

```
   building      varchar (15),
```

```
   room_number   varchar (7),
```

```
   time_slot_id  varchar (4),
```

```
  primary key (course_id, sec_id, semester, year),
```

```
  check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```



Referential Integrity

- ▶ **Referential Integrity**: the condition that ensure a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
- ▶ More generally, Let A be a set of attributes. Let R and S be two relations that contain attributes A and where A is the *primary key* of S . A is said to be a *foreign key* of R if for any values of A appearing in R these values also appear in S .
- ▶ Unlike *foreign-key constraints*, in general a **referential-integrity constraint** does not require A to be a primary key of S ; as a result, more than one tuple in S can have the same value for attributes A .



Cascading Actions in Referential Integrity

- ▶ When a *referential-integrity constraint is violated*, the normal procedure is to **reject the action** that caused the violation.
- ▶ However, a **foreign key** clause can **specify** that if a delete or update action on the referenced relation violates the constraint, then, instead of rejecting the action, the system must *take steps to change the tuple in the referencing relation to restore the constraint*.

```
create table course
( ...
  foreign key (dept_name) references department
    on delete cascade
    on update cascade,
  ... );
```



Cascading Actions in Referential Integrity

```
create table course
( ...
  foreign key (dept_name) references department
    on delete cascade
    on update cascade,
  ... );
```

- ▶ Because of the clause on **delete cascade** associated with the **foreign-key declaration**, if a delete of a tuple in *department* results in this referential-integrity constraint being violated, the system does not reject the delete.
- ▶ Instead, the **delete “cascades”** to the *course* relation, deleting the tuple that refers to the *department* that was deleted.



Cascading Actions in Referential Integrity

```
create table course
( ...
  foreign key (dept_name) references department
    on delete cascade
    on update cascade,
  ... );
```

- ▶ SQL also allows the **foreign key** clause to specify actions other than **cascade**:
 - ▶ The referencing field (here, *dept_name*) can be set to **null** (by using **set null** in place of cascade)
 - ▶ or to the **default** value for the domain (by using **set default**)



Integrity Constraint Violation During a Transaction

- ▶ **Transactions** may consist of several steps, and integrity constraints may be *violated temporarily* after one step, but a later step may remove the violation.

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```



Integrity Constraint Violation During a Transaction

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

- ▶ **How to insert a tuple without causing constraint violation?**
 - ▶ insert *father* and *mother* of a person before inserting *person*
 - ▶ OR, set *father* and *mother* to *null* initially, update after inserting all persons (not possible if *father* and *mother* attributes declared to be *not null*)
 - ▶ OR **defer constraint checking**



The deferred Clause

- ▶ The SQL standard allows a clause **initially deferred** *to be added to a constraint specification*; the constraint would then be checked at the end of a transaction, and not at intermediate steps.
- ▶ A constraint can alternatively be specified as **deferrable**.
- ▶ For constraints declared as deferrable, executing a statement

set constraints constraint-list deferred

as part of a transaction causes the checking of the specified constraints to be *deferred to the end of that transaction*.



Date and Time Types in SQL

- ▶ **Date:** A calendar date containing a (4 digit) year, month and date.
 - ▶ Example: **date** '2005-7-27'
- ▶ **Time:** The time of day, in hours, minutes and seconds.
 - ▶ Example: **time** '09:00:30' **time** '09:00:30.75'
 - ▶ **time(p)** : to specify the number of fractional digits for seconds (the default being 0)
- ▶ **Timestamp:** A combination of **date** and **time**.
 - ▶ Example: **timestamp** '2005-7-27 09:00:30.75'
- ▶ **Interval:** period of time
 - ▶ Example: **interval** '1' day



Date and Time Types in SQL

- ▶ **cast e as t** : convert a character string (or string valued expression) e to the type t , where t is one of **date**, **time**, or **timestamp**.
- ▶ **extract ($field$ from d)** : extract individual fields of a **date** or **time** value d , where $field$ can be one of **year**, **month**, **day**, **hour**, **minute**, or **second**.
- ▶ SQL defines several functions to get the current date and time
 - ▶ **current_date** returns the current date,
 - ▶ **current_time** returns the current time (with time zone)
 - ▶ **localtime** returns the current local time (without time zone)
 - ▶ **current_timestamp** returns timestamps(date plus time) with time zone
 - ▶ **localtimestamp** returns local date and time without time zone



Default Values

- ▶ SQL allows a **default value** to be specified for an attribute as illustrated by the following **create table** statement:

```
create table student
    (ID          varchar (5),
     name       varchar (20) not null,
     dept_name  varchar (20),
     tot_cred   numeric (3,0) default 0,
     primary key (ID));
```

- ▶ If no value is provided for the *tot_cred* attribute, its value is set to 0.
- ▶ An insert statement **without the value** for the *tot_cred* attribute:

```
insert into student(ID, name, dept_name)
    values ('12789', 'Newman', 'Comp. Sci.);
```



Index Creation

```
create table student  
  (ID varchar (5),  
   name varchar (20) not null,  
   dept_name varchar (20),  
   tot_cred numeric (3,0) default 0,  
   primary key (ID))
```

```
create index studentID_index on student(ID)
```

- ▶ Indices are data structures used to **speed up access** to records with specified values for index attributes.

- ▶ e.g. **select ***

- from** *student*

- where** *ID* = '12345'

- ▶ This can be executed by using the index to find the required record, **without looking at all records of *student***



Large-Object Types

- ▶ **Large objects** (photos, videos, CAD files, etc.) are stored as a *large object*:
 - ▶ **blob**: *binary large object* -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
 - ▶ **clob**: *character large object* -- object is a large collection of character data
 - ▶ When a query returns a large object, a **pointer** is returned rather than the large object itself.
 - ▶ The pointer can then be used to fetch the large object in small pieces, rather than all at once.
- ▶ Example: *book_review* **clob** (10KB)
 - image* **blob** (10MB)
 - movie* **blob** (2GB)



Authorization

- ▶ Forms of **authorization on parts of the database**
 - ▶ *Read* - allows reading, but not modification of data.
 - ▶ *Insert* - allows insertion of new data, but not modification of existing data.
 - ▶ *Update* - allows modification, but not deletion of data.
 - ▶ *Delete* - allows deletion of data.
- ▶ Forms of **authorization to modify the database schema**
 - ▶ *Index* - allows creation and deletion of indices.
 - ▶ *Resources* - allows creation of new relations.
 - ▶ *Alteration* - allows addition or deletion of attributes in a relation.
 - ▶ *Drop* - allows deletion of relations.



Authorization Specification in SQL

- ▶ The **grant statement** is used to confer authorization
 - grant** *<privilege list>*
 - on** *<relation name or view name>*
 - to** *<user list>*
- ▶ *<user list>* is:
 - ▶ **User-id**
 - ▶ **Public** (allows all valid users the privilege granted)
 - ▶ **Role**
- ▶ Granting a *privilege on a view* does **NOT** imply granting any privileges on the underlying relations.
- ▶ The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).



Privileges in SQL

- ▶ **select**: allows read access to relation, or the ability to query using the view
 - ▶ Example: grant users U_1 , U_2 , and U_3 **select** authorization on the *instructor* relation:

grant select on *instructor* to U_1 , U_2 , U_3

- ▶ **insert**: the ability to insert tuples
- ▶ **update**: the ability to update using the SQL update statement
 - ▶ gives users Amit and Satoshi **update** authorization on the *budget* attribute of the *department* relation:

grant update (*budget*) on *department* to Amit, Satoshi;

- ▶ **delete**: the ability to delete tuples.
- ▶ **all privileges**: used as a short form for all the allowable privileges



Revoking Authorization in SQL

- ▶ The **revoke statement** is used to revoke authorization.

revoke <privilege list>

on <relation name or view name>

from <user/role list>;

- ▶ Example: **revoke select on** *branch* **from** U_1, U_2, U_3
 - ▶ <**privilege list**> may be **all** to revoke all privileges the revokee may hold.
 - ▶ If <revokee-list> includes **public**, all users lose the privilege except those granted it explicitly.
 - ▶ If the same privilege was **granted twice** to the same user by different grantees, the user may retain the privilege after the revocation.
 - ▶ All privileges that **depend on the privilege** being revoked are also revoked.



Roles

- ▶ **Roles** can be created in SQL as follows: **create role** *instructor*;
- ▶ **Roles can be granted privileges** :
grant select on *takes* **to** *instructor*;
- ▶ **Roles can be granted to users**, as well as to other roles:
grant *instructor* **to** *Amit*;
create role *teaching_assistant*;
grant *teaching_assistant* **to** *instructor*;
(*Instructor* **inherits all privileges** of *teaching_assistant*)



Authorization on Views

create view *geo_instructor* **as**

(select *

from *instructor*

where *dept_name* = 'Geology');

grant select on *geo_instructor* **to** *geo_staff*

- ▶ Suppose that a *geo_staff* member issues

select *

from *geo_instructor*;

- ▶ A user who ***creates a view*** cannot be given update authorization on a view ***without having update authorization*** on the relations used to define the view.
- ▶ If a user creates a view on which no authorization can be granted, the system will ***deny the view creation request***.



Authorizations on Schema

- ▶ The SQL standard specifies that: only the *owner of the schema* can carry out any modification to the schema.
- ▶ However, SQL includes a **references privilege** that *permits a user to declare foreign keys when creating relations*.
- ▶ Allows user Mariano to create relations that reference the key *dept_name* of the *department* relation as a foreign key:
grant references (dept_name) on department to Mariano;



Transfer of Privileges

- ▶ By default, *a user/role that is granted a privilege is not authorized to grant that privilege to another user/role.*
- ▶ However, if we wish it does, we append the **with grant option** clause to the appropriate **grant** command.
- ▶ E.g. If we wish to allow Amit the select privilege on *department* and allow Amit to grant this privilege to others, we write:

grant select on *department* to Amit with grant option;



Transfer of Privileges

- ▶ The passing of a specific authorization from one user to another can be represented by an **authorization graph**.

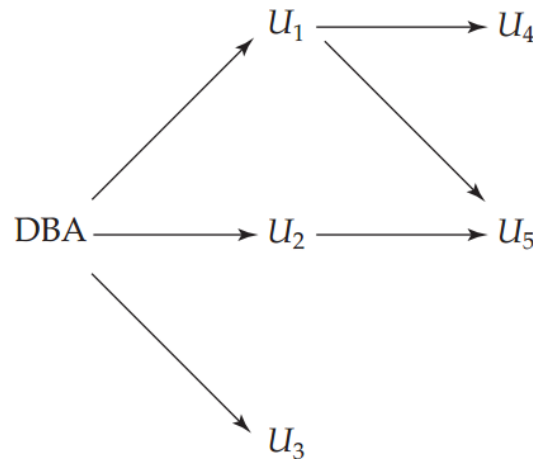


Figure 4.10 Authorization-grant graph ($U_1, U_2, \dots, U_5$ are users and DBA refers to the database administrator).

- ▶ A user has an authorization *if and only if* there is *a path from the root of the authorization graph* (the node representing the database administrator) down to the node representing the user.



Homework

► 4.1(c小题不用做), 4.6

4.1 Write the following queries in SQL:

- a. Display a list of all instructors, showing their ID, name, and the number of sections that they have taught. Make sure to show the number of sections as 0 for instructors who have not taught any section. Your query should use an outerjoin, and should not use scalar subqueries.
- b. Write the same query as above, but using a scalar subquery, without outerjoin.
- d. Display the list of all departments, with the total number of instructors in each department, without using scalar subqueries. Make sure to correctly handle departments with no instructors.

4.6 Complete the SQL DDL definition of the university database of Figure 4.8 to include the relations *student*, *takes*, *advisor*, and *prereq*.

